Frontend

Test Case	Test Name	Test Steps	Description / Expected Result	Status
1	Renders all form fields	1. Render <LoginForm onLoginSuccess={fn} />. 2. Query for email input, password input, Login button.	Email, password inputs and Login button present	✓ PASS
2	Empty email shows error	1. Click the Login button without filling any field.	Submitting with no email shows 'Email is required'	✓ PASS
3	Empty password shows error	1. Type a valid email. 2. Click Login without filling password.	Submitting with no password shows 'Password is required'	✓ PASS
4	No fetch on validation failure	1. Click Login with empty fields.	fetch() not called when form is invalid	✓ PASS
5	Fetch called with correct payload	1. Type 'User@Example.COM' in email field. 2. Type 'password123' in password field. 3. Click Login.	Email lowercased, sent to /api/v1/auth/login	✓ PASS
6	Token stored in localStorage	1. Submit valid credentials. 2. Check localStorage after submission.	localStorage.getItem('token') equals returned JWT	✓ PASS
7	onLoginSuccess called on success	1. Submit valid credentials. 2. Wait for async resolution.	Callback fires once after successful login	✓ PASS
8	Server error shown	1. Submit credentials. 2. Wait for response.	'Invalid email or password' shown on 401 response	✓ PASS
9	Network error shown	1. Submit valid credentials.	'Network Error' shown when fetch throws	✓ PASS
10	Button disabled while loading	1. Submit valid credentials. 2. Immediately inspect button state.	Submit button is disabled while fetch is pending	✓ PASS
11	Renders all form fields	1. Render <RegisterForm onRegistrationSuccess={fn} />. 2. Query for email, password, confirm password, Register button.	Email, password, confirm password, Register button present	✓ PASS
12	Empty email shows error	1. Click Register without filling any field.	'Email is required' shown on empty submit	✓ PASS
13	Short password shows error	1. Type email. 2. Type password shorter than 8 characters. 3. Click Register.	'at least 8 characters' shown for short password	✓ PASS
14	Password mismatch shows error	1. Fill email and password. 2. Type different value in confirm password. 3. Click Register.	'Passwords do not match' shown when fields differ	✓ PASS
15	No fetch on validation failure	1. Click Register with empty form.	fetch() not called when form is invalid	✓ PASS

16	Fetch called with correct payload	1. Fill 'User@Example.COM', 'password123', 'password123'. 2. Click Register.	Email lowercased, sent to /api/v1/auth/register	✓ PASS
17	Success message and redirect	1. Submit valid form. 2. Advance timers by 800ms.	'Registration successful' shown; onRegistrationSuccess fires after 800ms	✓ PASS
18	Form cleared on success	1. Submit valid form. 2. Wait for success state.	All fields empty after successful registration	✓ PASS
19	Server error shown	1. Submit valid form.	'User already exists' shown on 409 response	✓ PASS
20	Button disabled while submitting	1. Submit valid form. 2. Inspect button state.	Register button disabled while fetch pending	✓ PASS
21	Empty state message shown	1. Render <ImageGallery images={[]} />.	'No images to display' shown for empty array	✓ PASS
22	Empty state for undefined prop	1. Render <ImageGallery images={undefined} />.	'No images to display' shown when images is undefined	✓ PASS
23	Renders img per image	1. Render <ImageGallery images={[img1, img2]} />. 2. Query all <img> elements.	One <img> element rendered per image with url	✓ PASS
24	title used as alt text	1. Render gallery with image { url: '...', title: 'Knee X-ray' }.	img alt equals image.title	✓ PASS
25	Falls back to originalName	1. Render gallery with image { url: '...', originalName: 'scan.png' }.	img alt equals originalName when title absent	✓ PASS
26	Falls back to fileUrl	1. Render gallery with image { fileUrl: '/uploads/scan.png', title: 'Scan' }.	img src equals fileUrl when url absent	✓ PASS
27	No preview placeholder	1. Render gallery with image { id: '1', title: 'Mystery' }.	'No preview' shown when url and fileUrl both absent	✓ PASS
28	Label text rendered below image	1. Render gallery with image { url: '...', title: 'Knee Scan' }.	Title/name text visible below the image	✓ PASS
29	Renders dropzone text	1. Render <ImageUploader />.	'Drag & drop or click' text visible	✓ PASS
30	Shows drag-active text	1. Fire dragOver event on the dropzone element.	'Drop files here' shown on dragenter	✓ PASS
31	No upload button initially	1. Render <ImageUploader /> without dropping files.	Upload button absent before files are dropped	✓ PASS

32	Upload button after drop	1. Fire drop event with a PNG file. 2. Query for Upload button.	Upload button appears after files are dropped	✓ PASS
33	DICOM placeholder shown	1. Fire drop event with a .dcm file. 2. Query for DICOM label.	'DICOM' placeholder shown for .dcm files	✓ PASS
34	Full upload flow succeeds	1. Drop a PNG file. 2. Click Upload. 3. Wait for all three fetch calls.	presign → PUT → register called in order; onUploadSuccess fires	✓ PASS
35	Alert on presign failure	1. Drop a file. 2. Click Upload.	window.alert called when presign returns non-ok	✓ PASS
36	Button disabled while uploading	1. Drop a file. 2. Click Upload. 3. Inspect button state.	Upload button disabled while fetch is pending	✓ PASS
37	LoginPage — brand heading	1. Render <LoginPage onLoginSuccess={fn} onSwitchToRegister={fn} />.	'KneeVision' h1 rendered	✓ PASS
38	LoginPage — subtitle	1. Render LoginPage.	'Sign in to your account' text present	✓ PASS
39	LoginPage — form rendered	1. Render LoginPage.	Email and password inputs present inside page	✓ PASS
40	LoginPage — switch to register	1. Click the Register button.	onSwitchToRegister called when Register button clicked	✓ PASS
41	RegisterPage — brand heading	1. Render <RegisterPage onRegistrationSuccess={fn} onSwitchToLogin={fn} />.	'KneeVision' h1 rendered	✓ PASS
42	RegisterPage — subtitle	1. Render RegisterPage.	'Create your account' text present	✓ PASS
43	RegisterPage — form rendered	1. Render RegisterPage.	Email and confirm password inputs present inside page	✓ PASS
44	RegisterPage — switch to login	1. Click the Login button.	onSwitchToLogin called when Login button clicked	✓ PASS
45	UploadPage — brand heading	1. Render <UploadPage onLogout={fn} />.	'KneeVision' h1 rendered	✓ PASS
46	UploadPage — subtitle	1. Render UploadPage.	'Upload and review knee X-ray' text present	✓ PASS
47	UploadPage — logout button	1. Render UploadPage.	Logout button visible	✓ PASS

48	UploadPage — logout callback	1. Click the Logout button.	onLogout called when Logout clicked	✓ PASS
49	UploadPage — gallery heading	1. Render UploadPage.	'Gallery' heading present	✓ PASS
50	UploadPage — empty state	1. Render UploadPage with no images.	'No images to display yet' shown initially	✓ PASS
51	UploadPage — add mock image	1. Click 'Add Mock Image'. 2. Query for img element.	Mock image appears in gallery after clicking 'Add Mock Image'	✓ PASS
52	UploadPage — empty state hidden	1. Click 'Add Mock Image'. 2. Query for empty state text.	Empty state message gone after image added	✓ PASS

Backend

Test Case	Test Name	Test Steps	Description / Expected Result	Status
1	Returns a Buffer	1. Create a 100×100 RGB PNG buffer using sharp. 2. Call preprocessImage(buffer). 3. Inspect the returned value.	preprocessImage() output is a Buffer instance	✓ PASS
2	Outputs PNG by default	1. Call preprocessImage(buffer) with no format option. 2. Inspect output metadata using sharp(result).metadata().	Default format produces image/png metadata	✓ PASS
3	Converts image to grayscale	1. Create a colour PNG buffer. 2. Call preprocessImage(buffer). 3. Read output metadata with sharp.	Output has 1 channel (grayscale)	✓ PASS
4	No upscale for small images	1. Call preprocessImage(buffer, { width: 1024, height: 1024 }). 2. Read output metadata.	50x50 image stays ≤50px when target is 1024x1024	✓ PASS
5	Resizes large image to fit target	1. Call preprocessImage(buffer, { width: 512, height: 512 }). 2. Read output metadata.	2000x2000 image resized to ≤512x512 when target is 512	✓ PASS
6	Preserves aspect ratio	1. Call preprocessImage(buffer, { width: 200, height: 200 }). 2. Compute width/height ratio of output.	200x100 (2:1) input produces ~2:1 ratio output	✓ PASS
7	Outputs JPEG when format=jpeg	1. Call preprocessImage(buffer, { format: 'jpeg' }). 2. Read output metadata.	format:'jpeg' option produces jpeg metadata	✓ PASS
8	Outputs WebP when format=webp	1. Call preprocessImage(buffer, { format: 'webp' }). 2. Read output metadata.	format:'webp' option produces webp metadata	✓ PASS
9	Normalize option does not throw	1. Call preprocessImage(buffer, { normalize: true }). 2. Await the result.	normalize:true runs without error	✓ PASS
10	Throws on invalid buffer	1. Create Buffer.from('not-an-image'). 2. Call preprocessImage(badBuffer). 3. Await and expect rejection.	Non-image buffer rejects with 'Image preprocessing failed'	✓ PASS
11	Throws on null input	1. Call preprocessImage(null). 2. Await and expect rejection.	null input causes rejection	✓ PASS
12	Throws when MONGODB_URI missing	1. Delete process.env.MONGODB_URI. 2. Call connectToMongo(). 3. Await the promise.	connectToMongo() rejects with 'MONGODB_URI is not set'	✓ PASS
13	Connects and returns db instance	1. Set process.env.MONGODB_URI to a valid URI. 2. Call connectToMongo(). 3. Inspect return value.	Valid URI resolves to a db object	✓ PASS
14	Creates unique index on users.email	1. Call connectToMongo(). 2. Check that db.collection('users').createIndex was called.	createIndex called with {email:1}, {unique:true}	✓ PASS
15	Caches db on repeated calls	1. Call connectToMongo() twice. 2. Check client.connect call count.	client.connect called exactly once after two connectToMongo() calls	✓ PASS

16	getDb throws before connect	1. Import getDb from fresh module. 2. Call getDb() immediately.	getDb() throws 'MongoDB not connected' before connectToMongo()	✓ PASS
17	getDb returns db after connect	1. Call connectToMongo(). 2. Call getDb(). 3. Inspect return value.	getDb() returns db object after successful connect	✓ PASS
18	GET /health returns 200	1. Send GET request to /health.	Response body is {status:'ok'}	✓ PASS
19	Register — valid input returns 201	1. POST /api/v1/auth/register with { email: 'test@example.com', password: 'password123' }. 2. Inspect response.	Returns id, email, createdAt on valid body	✓ PASS
20	Register — missing email → 400	1. POST /api/v1/auth/register with { password: 'password123' } only.	Returns 400 with 'required' message	✓ PASS
21	Register — missing password → 400	1. POST /api/v1/auth/register with { email: 'test@example.com' } only.	Returns 400	✓ PASS
22	Register — invalid email → 400	1. POST /api/v1/auth/register with { email: 'not-an-email', password: 'password123' }.	Returns 400 with 'Invalid email format'	✓ PASS
23	Register — short password → 400	1. POST /api/v1/auth/register with { email: 'test@example.com', password: 'short' }.	Returns 400 with '8 characters' message	✓ PASS
24	Register — duplicate user → 409	1. POST /api/v1/auth/register with existing email.	Returns 409 with 'User already exists'	✓ PASS
25	Register — email normalized	1. POST with { email: 'USER@EXAMPLE.COM', password: 'password123' }. 2. Check response body email field.	USER@EXAMPLE.COM stored as user@example.com	✓ PASS
26	Login — valid credentials → 200	1. POST /api/v1/auth/login with correct email and password.	Returns JWT token and user object	✓ PASS
27	Login — empty body → 400	1. POST /api/v1/auth/login with empty body {}.	Returns 400	✓ PASS
28	Login — unknown user → 401	1. POST /api/v1/auth/login with unregistered email.	Returns 401 with 'Invalid email or password'	✓ PASS
29	Login — wrong password → 401	1. POST /api/v1/auth/login with correct email but wrong password.	Returns 401	✓ PASS
30	Presign — valid input → 200	1. POST /api/v1/uploads/presign with { filename: 'xray.png', contentType: 'image/png' }.	Returns uploadUrl, fileUrl, method:PUT	✓ PASS
31	Presign — missing filename → 400	1. POST /api/v1/uploads/presign with { contentType: 'image/png' } only.	Returns 400	✓ PASS

32	Presign — missing contentType → 400	1. POST /api/v1/uploads/presign with { filename: 'xray.png' } only.	Returns 400	✓ PASS
33	Images — valid input → 201	1. POST /api/v1/images with { fileUrl: '/uploads/test.png', originalName: 'xray.png', contentType: 'image/png' }.	Returns id, fileUrl, originalName, contentType	✓ PASS
34	Images — missing fileUrl → 400	1. POST /api/v1/images with { originalName: 'xray.png' } only.	Returns 400 with 'fileUrl is required'	✓ PASS